

Exercise 3:

Installation & configuration (ffprobe/ffmpeg)

I downloaded FFmpeg (has `ffmpeg` and `ffprobe`) using the official builds and added the `bin` folder to the system PATH so that I can call both tools in any shell. The code in Jupyter notebook dynamically resolves the binaries at run-time using `shutil.which(ffmpeg)` and `shutil.which(ffprobe)`. Communication is via `subprocess.run(...)` so any outputs (JSON of `ffprobe`, logs/stderr of `ffmpeg`) are captured and processed programmatically. This makes the setup portable and doesn't hard-code to an OS.

Analysis of application (logic and structure)

The notebook uses a deterministic pipeline: inspect, validate, convert (if necessary), re-validate, report. All requirements are centralized in a single source of truth, `FESTIVAL_SPEC`.

`get_video_info(path)`: `ffprobe` with `-print_format json` and specific `-show_entries` to get container name/bitrate as well as per-stream fields (codec, width/height, `avg_frame_rate` / `r_frame_rate`, display aspect ratio, stream bitrates, channels). The JSON is decoded into a dictionary. Missing fields are dealt with defensively and the first valid video/audio streams are selected.

`check_format(info)`: Checks probed fields against `FESTIVAL_SPEC`. Effective FPS is calculated out of `avg_frame_rate` (falling back to `r_frame_rate`) by parsing rational numbers like `30000/1001`. Aspect ratio is read when available in `display_aspect_ratio` otherwise calculated with a small tolerance based on width:height. Bitrates are scaled to integers to allow thresholds to be consistent. Any mismatch (wrong container/codec, fps not 25, resolution not 640x360, video bitrate not 2-5 Mb/s, audio bitrate >256 kb/s, channels !=2) is added to a human-readable list of problems.

`convert_video(src, dst)`: Uses `ffmpeg` to normalize all the properties in a single pass. The command imposes H.264 video and AAC audio, `-r 25`, resizes to 640x360, limits video bitrate to 2-5 Mb/s range, limits audio bitrate to 256 kb/s or less, and sets `-ac 2` to stereo. The target video bitrate is selected deterministically by clamping the nominal rate of the source to within the permitted range (rather than randomly selecting one of the endpoints). The output file has `_formatOK` name to it. The method returns a success flag and captures logs to report.

`check_converted_format(path)`: Re-runs the same validation on each newly produced `_formatOK` file to ensure that it passes the checks.

`generate_report(files, report_path)`: Goes through all the inputs and writes a concise per-file section (measured specs, issues in case any), and triggers conversion when `check_format` reports errors, and finally records the post-conversion validation result in TXT report.

Key Terms

Video Format (Container): The format in which the video data is stored along with audio, subtitles and other metadata.

Video Codec: The algorithm to compress and decompress video data.

Audio Codec: Like video codec, but it compresses and decompresses audio data.

Frame Rate: The rate at which the frames are shown (fps).

Aspect Ratio: The ratio of video width to video height.

Resolution: The video pixel dimensions.

Video Bit Rate: The number of data to express the video in a second.

Audio Bit Rate: The quantity of data employed to represent the audio in a second.

Audio Channels: The number of separate audio channels in the file.